



**Faculty of Engineering & Technology Electrical & Computer
Engineering Department
ENCS3390
Operating Systems
Project1 Report**

Prebread by:

Yara khattab

1210520

Instructor:

Dr. Abdel Salam Sayyad

Section: 2

Date: 3-5-2024

1- Abstract

The aim of this task is to learn about processes and threads management, using different approaches: Naive, Child Processes, and Threads. And to learn deeply about their performances and analysing each one execution time.

Table of Contents

Abstract.....	II
Table of Contents	III
Table of figures.....	IV
List of Tables.....	IV
Theory	1
Child Process	1
Threads	1
The API and functions.....	2
Results and Analysis.....	5
Naive approach	5
Multiprocessing approach	5
Multithreading approach.....	8
Conclusion	12

Table of figures

Figure 1:Child Processes	1
Figure 2:Threads.....	1
Figure 3:execution time for read from file.....	6
Figure 4:execution time for clculate the BMI code	6

List of Tables

Table 1:Naive approach results.....	5
Table 2:Multiprocessing approach results	5
Table 3:Multithreading approach results	8
Table 4:compares the performance.....	11

Theory

Child Process

A child process is created by a parent process in computing. The parent process can be thought of as the main process that spawns child or subprocesses to perform specific tasks. While a parent process can have multiple child processes, each child process has only one parent. When a child process is created, it inherits most of its parent's attributes, such as file descriptors and memory space. In the C language, child processes are typically created using the `fork()` function. It is essential for the parent process to wait for its child processes to complete their work, which can be achieved by calling the `wait()` function. This synchronization ensures proper coordination between parent and child processes in a multi-process environment.

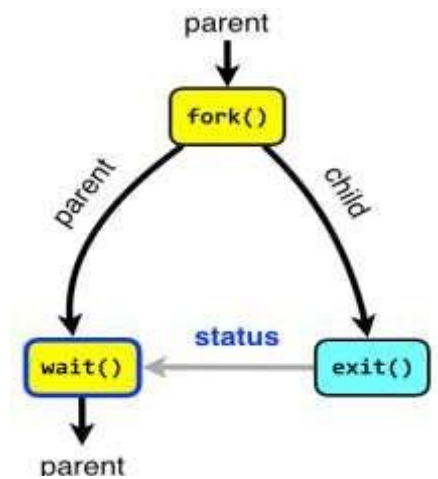


Figure 1:Child Processes

Threads

Within a program, a Thread is a separate execution path. It is a lightweight process that the operating system can schedule and run concurrently with other threads. The operating system creates and manages threads, and they share the same memory and resources as the program that created them. This enables multiple threads to collaborate and work efficiently within a single program.

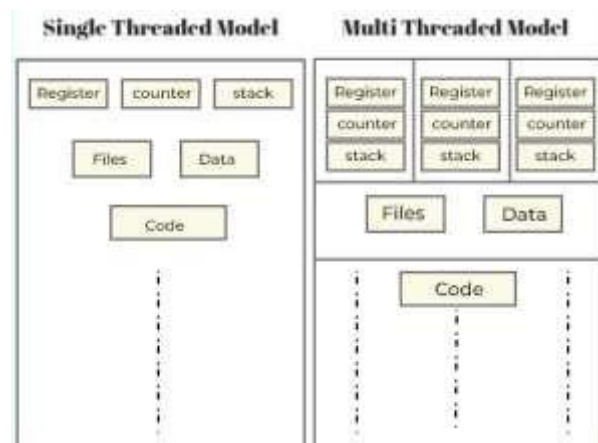


Figure 2:Threads

A thread is a single sequence stream within a process.

Threads are also called lightweight processes as they possess some of the properties of processes. Each thread belongs to exactly one process. In an operating system that supports multithreading, the process can consist of many threads. But threads can be effective only if CPU is more than 1 otherwise two threads have to context switch for that single CPU

How I achieved the multiprocessing and multithreading requirements. The API and functions that I used.

For both multiprocessing and multithreading:

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#include <time.h>
#include <pthread.h>
```

first I used these libraries in the code:

- 1- `stdio.h` → for input/output operations.
- 2- `stdlib.h` → for memory management and process control.
- 3- `sys/wait.h` → for process termination status.
- 4- `unistd.h` → for POSIX OS API.
- 5- `time.h` → for the time execution.

```
// begin time
struct timespec BeginTime;
timespec_get(&BeginTime, TIME_UTC);

// end time
struct timespec EndTime;
timespec_get(&EndTime, TIME_UTC);

double timespentinexecution=(EndTime.tv_sec - BeginTime.tv_sec)+(EndTime.tv_nsec - BeginTime.tv_nsec)/1000000000.0; // to get the execution time
printf("The Execution Time using Multiprocessing approach = %lf second\n", timespentinexecution); // to print the execution time in seconds
```

This code measures the execution time of a section of a program in seconds. It records the start and end times using `timespec_get`, then calculates the execution time by subtracting the start time from the end time.

For multiprocessing:

```
// Create pipes
int fd[numberOfprocessneed][2]; // Four pipe file descriptors
for (int i = 0; i < numberOfprocessneed; i++)
{
    if (pipe(fd[i]) == -1)
    {
        perror("Error, can not creat the pipe");
        exit(EXIT_FAILURE);
    }
}
```

Pipes in programming are used to allow one process to send data to another process in a unidirectional flow. They are commonly used for inter-process communication (IPC), acting as a conduit for data between two points in a system. so in the code above I create an array of pipe file descriptors (fd) to facilitate communication between processes.

```
pid_t pid= fork();// creat new child
```

The line above creates a new process, known as a child process, which runs concurrently with the process that made the call (the parent process). The fork() function returns a process identifier (PID):0 to the child process and the child's PID to the parent, or -1 if the fork creation is failed. This allows the program to branch into two parallel executions.

```
// parent Wait for all child processes to complete
for (int i = 0; i < 4; ++i) {
    wait(NULL);
}
```

This code synchronizes the parent process with its child processes, allowing them to finish their work before the parent continues. It's a multi-process programming, in the code we can see that the parent process waits for four child processes to finish their execution.

The for loop runs four iterations (from i = 0 to i < 4 → depending on how many process I have). Inside each iteration, the wait(NULL) function is called to wait for any child process to terminate. Then after the parent process ensure that all it's child process finish there work ,it will work.

For multithreading:

```
pthread_mutex_t lock; // Mutex for protecting shared resources
```

The line above defines a mutex, which is a lock that threads can use to ensure they don't access the same variable or part of memory at the same time. It's used to prevent "race conditions" and accessing the same resource at the same time, ensuring that only one thread can modify shared data at any moment.

```
pthread_mutex_unlock(&lock);
```

The line above releases a mutex previously locked by the same thread, allowing other threads to acquire it and proceed with their execution. It's essential for fair and efficient sharing of resources in multithreaded applications.

```
if (pthread_mutex_init(&lock, NULL) != 0) // Initialize the mutex
{
    printf("Mutex init has failed\n");
    return 1;
}
```

The line above initializes a mutex and it's a function from the POSIX threads library (pthread). It creates a mutex and sets its initial state. The first argument (&lock) is the address of the mutex variable to be initialized and the second argument (NULL) specifies the default attributes for the mutex.

```
pthread_t threads[number_of_thread]; // Create threads
```

The line above declares an array of thread identifiers (threads) of type pthread_t. The size of the array is determined by number_of_thread, and it's used to manage multiple threads in a program, allowing for parallel execution of tasks.

```
pthread_join(threads[m], NULL);
```

pthread_join synchronizes the parent thread with a specific child thread, ensures that the parent thread doesn't continue until the specified child thread has completed its task.

```
pthread_mutex_destroy(&lock); // Destroy the mutex
```

The line above shows a function call that deactivates a mutex, named lock, that was previously initialized. This is done when the mutex is no longer needed. It's a good practice to destroy a mutex once it's no longer in use to avoid resource leaks in a program.

Results and Analysis

Naive approach

The naive approach in the code handles a sequential and straightforward manner. It does not use parallel computing techniques.

Results of naive approach (time in seconds):

Run1	Run2	Run3	Run4	Run5	Avg
0.000230	0.000234	0.000271	0.000198	0.005346	0.0012558

Table 1:Naive approach results

Performance=1/0.0012558 =**796.30**

Multiprocessing approach

In the Multiprocessing approach, the data is distributed among multiple child processes created using “fork()” function. Each process calculates a portion of the data, potentially enhancing performance on multi-core systems. However, this method involves the overhead of inter-process communication and process management.

Results of child processes approach (time in seconds):

Processes	Run1	Run2	Run3	Run4	Run5	Avg
2	0.007658	0.01241	0.007980	0.006372	0.009555	0.008795
4	0.008056	0.007584	0.007025	0.006492	0.005638	0.006959
6	0.007299	0.010126	0.006754	0.007395	0.008205	0.007955

Table 2:Multiprocessing approach results

Performance (2 processes) = 1/0.008795=**113.7**

Performance (4 processes) = 1/0.006959=**143.69**

Performance (6 processes) = 1/0.007955= **125.7**

Amdahl's law for Multiprocessing approach:

What percentage is the serial part of your code?

The code have three parts the first part is for reading the data from the file, second part is to calculate the BMI for each person and the third part to calculate the average BMI. So the serial part of the code is the first part + the third part.

The serial part:

1- First part : reading from the file

The code for do it:

```
int main()
{
    // begin time
    struct timespec BeginTime;
    timespec_get(&BeginTime, TIME_UTC);

    int numOfpeopleinFile=500;
    struct ThePersoninformation person [numOfpeopleinFile];

    ReadfromFile(numOfpeopleinFile, person);

    // end time
    struct timespec EndTime;
    timespec_get(&EndTime, TIME_UTC);

    double timespentinexecution=(EndTime.tv_sec - BeginTime.tv_sec)+(EndTime.tv_nsec - BeginTime.tv_nsec)/1000000000.0; // to get the execution time
    printf("The Execution Time for read from the file = %lf second\n", timespentinexecution); //to print the execution time in seconds

    return 0;
}
```

Figure 3: execution time for read from file

The time result for reading → 2 process=0.000373 sec

→ 4 process=0.000304 sec

→ 6 process=0.000292 sec

2- Second part: calculate the average BMI

The code to do it

```
// begin time
struct timespec BeginTime;
timespec_get(&BeginTime, TIME_UTC);

// Parent process
double SumOfallBMI = 0.0;
for (int i = 0; i < numberOfprocessneed; i++) {
    close(fd[i][1]); // Close write end
    for (int j = i * TheDataForeachprocess; j < (i + 1) * TheDataForeachprocess; j++) {
        //double BMI;
        read(fd[i][0], &BMI, sizeof(double));
        SumOfallBMI += BMI;
    }
    close(fd[i][0]);
}

// Calculate and print the average BMI using Multiprocessing approach
double averageBMI = SumOfallBMI / numOfpeopleinFile;
printf(" The Average BMI for Multiprocessing approach: %.2lf\n", averageBMI);

// parent Wait for all child processes to complete
for (int i = 0; i < 2; ++i) {
    wait(NULL);
}

// end time
struct timespec EndTime;
timespec_get(&EndTime, TIME_UTC);

double timespentinexecution=(EndTime.tv_sec - BeginTime.tv_sec)+(EndTime.tv_nsec - BeginTime.tv_nsec)/1000000000.0; // to get the execution time
printf("The Execution Time for calculate the BMI = %lf second\n", timespentinexecution); //to print the execution time in seconds
```

Figure 4: execution time for calculate the BMI code

The time result for calculate BMI → 2 process=0.005532 sec

→ 4 process=0.005431 sec

→ 6 process=0.004557 sec

The total time avg BMI → 2 process=0.008795 sec

→ 4 process=0.006959 sec

→ 6 process=0.007955 sec

percentage *serial part* = ((T read file+T calculate BMI) / avg BMI)*100%

→ 2 process → $\frac{0.000373+0.005532}{0.008795} * 100\% = 67\%$

→ 4 process → $\frac{0.000304 + 0.005431}{0.006959} * 100\% = 82\%$

→ 6 process → $\frac{0.000292+0.004557}{0.007955} * 100\% = 72\%$

Amdahl's law → $\text{speedup} = \frac{1}{s + \frac{(1-s)}{N}}$

→ 2 process → speedup=1.2

→ 4 process → speedup=1.1

→ 6 process → speedup=1.3

From the results above we can see that the 6 process is the optimal.

Multithreading approach

The Multithreading approach utilizes multithreading for parallel computation, dividing the program into tasks among several threads. These threads are joinable, which means the main thread waits for all threads to complete, allowing for easier synchronization and result collection.

Results of joinable threads approach (time in seconds):

Threads	Run1	Run2	Run3	Run4	Run5	Avg
2	0.002746	0.003089	0.002065	0.003296	0.003228	0.002884
4	0.001654	0.001903	0.001888	0.001906	0.001864	0.001843
6	0.001995	0.002014	0.002135	0.002055	0.002001	0.002040

Table 3:Multithreading approach results

Performance (2 threads) = $1/0.002884 = 346.7$

Performance (4 threads) = $1/0.001843 = 542.5$

Performance (6 threads) = $1/0.002040 = 490.9$

Amdahl's law for Multithreading approach:

What percentage is the serial part of your code?

The code have three parts the first part is for reading the data from the file, second part is to calculate the BMI for each person and the third part to calculate the average BMI. So the serial part of the code is the first part + the third part.

The serial part:

1-First part : reading from the file

The time result for reading → **2 threads** =0.000328 sec

→ **4 threads** =0.000321 sec

→ **6 threads** =0.000316 sec

2-Second part: calculate the average BMI

The time result for calculate BMI → **2 threads** =0.000193 sec

→ **4 threads** =0.000160 sec

→ **6 threads** =0.000140 sec

The total time avg BMI → **2 threads** =0.002884 sec

→ **4 threads** =0.001843 sec

→ **6 threads** =0.002040 sec

percentge *serial part* = ((T read file+T calculate BMI) / avg BMI)*100%

→2 threads → $\frac{0.000328+0.000193}{0.002884} * 100\% = 18\%$

→4 threads → $\frac{0.000321+0.000160}{0.001843} * 100\% = 26\%$

→6 threads → $\frac{0.000316+0.000140}{0.002040} * 100\% = 22\%$

Amdahl's law → speedup= $\frac{1}{s + \frac{(1-s)}{N}}$

→2 threads → speedup=1.6

→4 threads → speedup=2.2

→6 threads → speedup=2.8

From the results above we can see that the 6 threads is the optimal.

Summarization

Approach	Performance
Naive	796.30
Multiprocessing (2)	113.7
Multiprocessing (4)	143.69
Multiprocessing (6)	125.7
Multithreading (2)	346.7
Multithreading (4)	542.5
Multithreading (6)	490.9

Table 4:compares the performance

Conclusion

In summary, this project showed the efficiency of parallel computing in processing large datasets by comparing three different approaches to calculate average BMI, The naive approach , multiprocessing approach and multithreading approach. The multiprocessing and multithreading approaches utilized system cores to expedite computation. From the results above I got that the naive approach is the most effective method with highest performance, while the performance for multiprocessing approach and multithreading approach it's change according to the number of process(childs) or threads we have ,so in my code I found that the 6 process and 6 threads are the optimal according to amdal's law.